OmniSort AI

Complete Function Reference

Privacy-first AI file organizer · Python 3.11+

Generated June 02, 2026

18 modules · 60+ functions documented

File Watching | PII Detection | Classification | Embeddings | REST API | SQLite

Table of Contents

ENTRY POINT

→ main.py — Application Entry Point

CORE PIPELINE

→ file_watcher.py — Core Orchestrator

CLASSIFIERS

→ image_classifier.py — Image & Document NLP

→ llm_classifier.py — GPT-4o-mini Fallback

→ rules_engine.py — Custom Rules Engine

DETECTION

→ sensitive.py — On-Device PII Detector

→ duplicate_detector.py — SHA-256 Duplicate Detection

TEXT EXTRACTION

→ ocr_extractor.py — OCR Text Extraction

→ pdf_process.py — PDF Text Extraction

→ doc_process.py — DOCX Text Extraction

→ text_process.py — Plain Text Reader

ROUTING & STORAGE

→ policy_engine.py — Routing Flag Setter

→ file_organizer.py — File Mover

→ db.py — SQLite Data Layer

INTELLIGENCE

→ embedding.py — Semantic Embedding

INFRASTRUCTURE

→ api.py — FastAPI REST + WebSocket Server

→ metrics.py — Observability Metrics

→ logger.py — Structured Logger

Processing Pipeline — End-to-End Flow

Every file dropped into a watched folder passes through these 15 steps in order. Steps 2 and 5 are hard PII gates — if PII is detected, steps 6–14 are skipped entirely and the LLM is never called.

#	Function / Module	What it does	Exits pipeline if...
1	`_wait_for_file_ready()`	Wait for file to finish downloading	Timeout → skip
2	`SensitiveDetector (filename)`	PII in filename before file is opened	PII found → Sensitive/
3	`PDFProcessor / DocxProcessor / TextProcessor / OCRExtractor`	Extract text	—
4	`OCRExtractor.extract_text_from_pdf()`	OCR fallback for scanned PDFs	—
5	`SensitiveDetector (content)`	PII in extracted text	PII found → Sensitive/
6	`RulesEngine.match()`	Custom keyword rule match	Match → custom folder
7	`classify_document()`	Keyword NLP: Invoices / Resumes / Legal	Not 'Documents' → done
8	`LLMClassifier.classify()`	GPT-4o-mini fallback classification	Returns category
9	`DuplicateDetector.calculate_file_hash()`	SHA-256 hash	—
10	`DuplicateDetector.is_duplicate()`	Hash lookup in SQLite [per-hash lock]	Match → Duplicates/
11	`PolicyEngine.apply_policies()`	Set is_sensitive / is_duplicate flags	—
12	`FileOrganizer.organize_file()`	shutil.move to category subfolder	—
13	`embed_text()`	384-dim embedding stored in SQLite	Error → NULL (graceful)
14	`db.insert_file()`	Write full record to SQLite	Error → logged, file safe
15	`event_queue.put()`	Broadcast JSON event to WebSocket clients	—

main.py — Application Entry Point

backend/main.py

Bootstraps the entire application. Loads YAML configuration, starts the FileWatcher (file-system monitoring), then launches the FastAPI server in a background daemon thread. Blocks on the watchdog observer so the process stays alive until Ctrl-C.

● load_config

```
load_config() -> dict
```

Reads configs/settings.yaml relative to the module file using os.path.abspath, then returns the parsed YAML as a Python dict. Called once at startup.

RETURNS

dict — full settings.yaml contents

■ *Uses abspath so it works regardless of the working directory the process was started from.*

● run_api

```
run_api(host: str, port: int) -> None
```

Starts the uvicorn ASGI server for the FastAPI app. Always called in a daemon thread from __main__ to avoid blocking the watchdog observer.

PARAMETERS

host — Interface to bind, e.g. '127.0.0.1'

port — TCP port, e.g. 8000

RETURNS

None — runs forever until the process exits

■ *log_level='warning' suppresses uvicorn per-request access logs.*

file_watcher.py — Core Orchestrator

backend/watcher/file_watcher.py

The heart of OmniSort. Registers watchdog observers for every configured watch folder and runs the full 15-step classification pipeline for each new file. Enforces all safety invariants: thread-pool cap, per-path deduplication, per-hash atomic locking, output-folder re-trigger guard, and PII-before-LLM ordering.

● _load_config

```
_load_config() -> dict
```

Reads configs/settings.yaml and applies two Docker environment variable overrides: OMNISORT_WATCH_FOLDER replaces watch_folders, OMNISORT_OUTPUT_FOLDER replaces output_folder.

RETURNS

dict — merged config (YAML + env vars)

■ *Env var overrides let the Docker image mount any host path without editing YAML.*

● _acquire_lock

```
_acquire_lock() -> bool
```

Opens /tmp/omnisort.lock and acquires an exclusive non-blocking fcntl flock. Returns True if this process owns the lock, False if another instance is running.

RETURNS

bool — True = lock acquired

■ *The lock is freed automatically by the OS when the process exits.*

● _should_skip

```
_should_skip(file_path: str) -> bool
```

Returns True for files that must never be processed: browser download temp files (.crdownload, .part, .tmp, .download, .!ut), macOS metadata (.DS_Store), and browser-internal dotfiles (.com.brave.*, .com.google.*).

PARAMETERS

file_path — Absolute or relative path to the file

RETURNS

bool — True means skip entirely

■ *Checked in on_created, on_moved, and _scan_existing.*

● FileWatcher.__init__

```
__init__(self, folder_path: str = None)
```

Initialises all subsystems: loads config, resolves watch folders, creates ThreadPoolExecutor(max_workers=4), instantiates all classifier/detector/organizer objects, sets up the watchdog Observer, and calls db.init_db().

PARAMETERS

folder_path — Optional single folder, overrides settings.yaml watch_folders

RETURNS

None

■ *ThreadPoolExecutor(max_workers=4) prevents thread explosion when hundreds of files land simultaneously.*

■ *_hash_locks + _hash_locks_lock guard concurrent duplicate detection for identical files.*

● FileWatcher.start

```
start(self) -> None
```

Acquires the fcntl process lock, schedules one watchdog observer per watch folder, starts the observer, then spawns _scan_existing in a daemon thread.

RETURNS

None

■ *Raises RuntimeError if another OmniSort instance already holds the lock.*

● FileWatcher.stop

```
stop(self) -> None
```

Stops and joins the watchdog observer, shuts down the thread pool (wait=False), and logs the stop event.

RETURNS

None

● FileWatcher._get_hash_lock

```
_get_hash_lock(self, file_hash: str) -> threading.Lock
```

Returns a per-hash threading.Lock, creating one if it does not exist. Used to make the duplicate-check + file-move + DB-write sequence atomic, preventing two threads processing identical files from both seeing is_duplicate=False.

PARAMETERS

file_hash — SHA-256 hex digest

RETURNS

threading.Lock for this specific hash

■ *Access to _hash_locks itself is protected by _hash_locks_lock to prevent dict mutation races.*

● FileWatcher._scan_existing

```
_scan_existing(self) -> None
```

Startup scan (runs after a 1-second delay). Iterates all watch folders, skips directories, output-folder files, and _should_skip paths. Calls _process_file for everything else, ensuring files dropped while OmniSort was offline are caught.

RETURNS

None — runs in a daemon thread

■ *1-second sleep prevents a race with on_created firing for the same file.*

● FileWatcher.on_created

```
on_created(self, event: FileCreatedEvent) -> None
```

Watchdog callback for new files. Guards against directories, skip-list paths, and output-folder files. Submits _process_file to the thread pool.

PARAMETERS

event — watchdog.events.FileCreatedEvent

RETURNS

None

● FileWatcher.on_moved

```
on_moved(self, event: FileMovedEvent) -> None
```

Watchdog callback for file renames. Uses event.dest_path. Critical for AirDrop: iOS writes a .part temp file then renames it to the final extension — on_moved catches this rename; on_created would miss it.

PARAMETERS

event — watchdog.events.FileMovedEvent

RETURNS

None

■ *Without on_moved, all AirDropped files would be silently ignored.*

● FileWatcher._wait_for_file_ready

```
_wait_for_file_ready(self, file_path: str) -> bool
```

Polls os.path.getsize() every 500 ms until the file size stabilises across two consecutive reads AND size > 0. Returns True when ready, False on timeout. Prevents processing a file still being written by a browser or download manager.

PARAMETERS

file_path — Absolute path to the file

RETURNS

bool — True if ready, False if timed out

■ *Empty files (size=0) always time out and are skipped silently.*

● FileWatcher._get_type

```
_get_type(self, ext: str) -> str
```

Maps a lowercased extension to a type key: 'image', 'pdf', 'docx', 'text', 'video', 'audio', 'archive', or 'other'.

PARAMETERS

ext — Lowercased extension with dot, e.g. '.pdf'

RETURNS

str — type key used to select the correct processor

● FileWatcher._process_file

```
_process_file(self, file_path: str) -> None
```

The complete 15-step pipeline (runs in a thread pool worker): 1. Dedup guard: skip if already in _processing set. 2. _wait_for_file_ready: abort on timeout. 3. Determine file type. 4. Filename PII check (before file opened). 5. Type-specific text extraction (image/pdf/docx/text). 6. Scanned-PDF OCR fallback if PyPDF2 returns empty text. 7. Content PII check. 8. Merge filename+content PII — if any PII, skip steps 9-11. 9. Custom rules (RulesEngine.match). 10. Keyword NLP (classify_document). 11. LLM fallback (LLMClassifier.classify) if still 'Documents' and no PII. 12. SHA-256 hash. 13. Per-hash lock acquired. 14. Duplicate check + PolicyEngine + FileOrganizer.organize_file (all under lock). 15. embed_text + db.insert_file (DB write isolated in try/except) + event broadcast.

PARAMETERS

file_path — Path from the watchdog event

RETURNS

None

■ *Priority: Sensitive > Duplicate > Custom Rule > NLP > LLM.*

■ *Per-hash lock makes steps 13-15 atomic to prevent concurrent duplicate miss.*

■ *DB failure is logged but the already-moved file is never lost.*

---

image_classifier.py — Image & Document NLP

backend/classifier/image_classifier.py

Two classifiers: ImageClassifier uses filename patterns and pixel dimensions; classify_document() uses keyword matching on text. No ML model — deterministic, zero-latency, zero network calls.

ImageClassifier.predict

```
predict(self, image_path: str) -> str
```

Classifies an image as 'Screenshots' or 'Photos'. Checks filename against SCREENSHOT_PATTERNS (screenshot, screen_, capture, snip), then checks pixel dimensions against known screen resolutions. Falls back to 'Photos'.

PARAMETERS

image_path — Absolute path to image file

RETURNS

str — 'Screenshots' or 'Photos'

- *SCREENSHOT_WIDTHS: 1280 1366 1440 1920 2560 3840 2732 2388*
- *SCREENSHOT_HEIGHTS: 720 768 800 900 1024 1080 1440 2160 2048 2732*
- *PIL errors are silently ignored — corrupt images fall to 'Photos'.*

classify_document

```
classify_document(text: str) -> str
```

Stage-3 keyword NLP for text documents. Lowercases text and checks three keyword lists. Returns the first matching category or 'Documents'.

PARAMETERS

text — Full extracted document text

RETURNS

str — 'Invoices', 'Resumes', 'Legal', or 'Documents'

- *Invoices: invoice, bill, receipt, total amount, due date, payment due*
- *Resumes: resume, curriculum vitae, cv, objective, work experience, references*
- *Legal: contract, agreement, terms and conditions, hereby, whereas, shall*

llm_classifier.py — GPT-4o-mini Fallback

backend/classifier/llm_classifier.py

Stage-4 classifier. Only called when PII gate passed, no custom rule matched, and keyword NLP returned 'Documents'. Sends first 2,000 characters to GPT-4o-mini. Degrades gracefully to 'Documents' on any error.

● LLMClassifier.__init__

```
__init__(self)
```

Initialises the OpenAI client using the OPENAI_API_KEY environment variable. Client is created once and reused across all classify() calls.

RETURNS

None

- *If OPENAI_API_KEY is missing, all calls raise and return 'Documents'.*

● LLMClassifier.classify

```
classify(self, text: str) -> str
```

Truncates text to 2,000 chars, returns 'Documents' if blank. Sends a chat completion to gpt-4o-mini with a strict system prompt constraining the response to one of 8 categories. Validates the returned string; returns 'Documents' on validation failure or any exception.

PARAMETERS

text — Full extracted document text

RETURNS

str — one of: Invoices, Resumes, Legal, Medical, Financial, Academic, Documents, Other

- *max_tokens=16 — only one word needed in the response.*
- *Any network/quota exception silently returns 'Documents'.*
- *Called inside metrics.record_llm_call() timing block in file_watcher.*

sensitive.py — On-Device PII Detector

backend/sensitive_detection/sensitive.py

Regex-based PII detector. Runs twice per file: on the filename before the file is opened, and on extracted text before any NLP or LLM call. Pure Python re — no model, no network. A single match routes to Sensitive/ and blocks all downstream API calls.

● SensitiveDetector.__init__

```
__init__(self)
```

Compiles three regexes at construction: email (RFC-5321), phone (NNN-NNN-NNNN), SSN (NNN-NN-NNNN). Compiled once for efficiency.

RETURNS

None

- *email: \b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b*
- *phone: \b\d{3}[-.]?\d{3}[-.]?\d{4}\b*
- *ssn: \b\d{3}[-]?\d{2}[-]?\d{4}\b*

● SensitiveDetector.detect_sensitive_info

```
detect_sensitive_info(self, text: str) -> dict
```

Runs all three compiled patterns using findall(). Returns a dict mapping pattern name to list of matches. An empty dict means no PII found. Called on both filename and content; results merged with {**filename_pii, **content_pii}.

PARAMETERS

text — String to scan — filename or document text

RETURNS

dict — e.g. {'email': ['a@b.com'], 'ssn': ['123-45-6789']} or {}

- *Empty dict is falsy — file_watcher uses 'if sensitive_info' as the PII gate.*

---

rules_engine.py — Custom Rules Engine

backend/rules/rules_engine.py

User-defined keyword-to-folder routing loaded from custom_rules in settings.yaml. Runs after PII detection but before NLP/LLM — zero API cost, instant. A matching rule short-circuits the entire classification stack.

● RulesEngine.__init__

```
__init__(self, rules: list)
```

Accepts rule dicts from settings.yaml. Each has 'folder' (str) and 'keywords' (list). Keywords are lowercased at construction to avoid repeated case-folding.

PARAMETERS

rules — List of {'folder': str, 'keywords': [str]} from settings.yaml

RETURNS

None

■ *Empty list or None produces a no-op engine that always returns None.*

● RulesEngine.match

```
match(self, text: str, filename: str = '') -> str | None
```

Concatenates filename + text, lowercases the haystack, checks each rule's keywords using 'in'. Returns the folder name of the first matching rule, or None.

PARAMETERS

text — Extracted document text

filename — Basename of file

RETURNS

str — folder name (e.g. 'Bank') or None

■ *Called with 'if not sensitive_info' guard — PII always wins over custom rules.*

■ *First matching rule wins; order rules from most specific to most general.*

---

duplicate_detector.py — SHA-256 Duplicate Detection

backend/duplicate_detection/duplicate_detector.py

Content-based duplicate detection. Works across filenames — 'doc.pdf' and 'doc_copy.pdf' with identical content are correctly identified as duplicates. Both methods are called inside the per-hash lock in file_watcher to prevent TOCTOU races.

● DuplicateDetector.calculate_file_hash

```
calculate_file_hash(self, file_path: str) -> str
```

Reads the file in 4 KB binary chunks and feeds them to a SHA-256 hasher. Streaming avoids loading large files into memory. Returns the 64-char hex digest.

PARAMETERS

file_path — Absolute path to file

RETURNS

str — 64-character SHA-256 hex digest

■ *4,096-byte chunks balance memory usage and system-call overhead.*

● DuplicateDetector.is_duplicate

```
is_duplicate(self, file_hash: str) -> bool
```

Queries SQLite files table for any row with matching file_hash. Returns True if found (seen before). Must be called inside the per-hash threading.Lock.

PARAMETERS

file_hash — SHA-256 hex digest to look up

RETURNS

bool — True if this hash exists in the database

■ *The lock is owned by FileWatcher._get_hash_lock(), not this class.*

ocr_extractor.py — OCR Text Extraction

backend/ocr/ocr_extractor.py

Wraps pytesseract for three scenarios: file-path images, in-memory PIL images, and scanned PDFs with no text layer. Auto-discovers the Tesseract binary.

● _find_tesseract

```
_find_tesseract(config_path: str = None) -> str
```

Tries the configured path, then /usr/local/bin, /opt/homebrew/bin (Apple Silicon), /usr/bin, then shutil.which('tesseract'). Returns the first found path.

PARAMETERS

config_path — Optional path from settings.yaml

RETURNS

str — path to the tesseract binary

■ *Covers both Intel (/usr/local) and Apple Silicon (/opt/homebrew) Mac installs.*

● OCRExtractor.__init__

```
__init__(self, tesseract_path: str = None)
```

Calls _find_tesseract() and assigns the result to pytesseract.tesseract_cmd.

PARAMETERS

tesseract_path — Optional path from config

RETURNS

None

● OCRExtractor.extract_text

```
extract_text(self, image_path: str) -> str
```

Opens image with PIL.Image.open() and passes to pytesseract.image_to_string(). Returns stripped text. Used for .jpg, .png, .webp, etc.

PARAMETERS

image_path — Absolute path to image file

RETURNS

str — OCR text, stripped

● OCRExtractor.extract_text_from_image

```
extract_text_from_image(self, image: PIL.Image) -> str
```

Same as extract_text but accepts an in-memory PIL Image object. Used by extract_text_from_pdf_page after rendering a PDF page to a pixmap.

PARAMETERS

image — PIL.Image object, e.g. a rendered PDF page

RETURNS

str — OCR text, stripped

● OCRExtractor.extract_text_from_pdf_page

```
extract_text_from_pdf_page(self, pdf_path: str, page_index: int = 0) -> str
```

OCR fallback for scanned PDFs. Opens with PyMuPDF (fitz), renders page to a 150-DPI pixmap, converts to PIL RGB image, then calls extract_text_from_image(). Called when PyPDF2 returns empty text (medical reports, tax forms, image-only PDFs).

PARAMETERS

pdf_path — Absolute path to PDF

page_index — Zero-based page number, default 0

RETURNS

str — OCR text from the rendered page, or '' on any error

■ *150 DPI balances OCR accuracy vs rendering speed.*

■ *All exceptions silently return '' so the pipeline never crashes.*

pdf_process.py — PDF Text Extraction

backend/processor/pdf_process.py

Extracts text and metadata from PDFs with text layers using PyPDF2.

● PDFProcessor.process

```
process(self, file_path: str) -> tuple[str, dict]
```

Opens the PDF in binary mode, iterates all pages, concatenates extract_text(), reads reader.metadata (title, author, creator). Returns (full_text, metadata).

PARAMETERS

file_path — Absolute path to PDF

RETURNS

tuple — (str: concatenated text, dict: PDF metadata)

- *Raises ValueError on error — caught by file_watcher's outer try/except.*
- *Empty text triggers OCRExtractor.extract_text_from_pdf_page() fallback in file_watcher.*

doc_process.py — DOCX Text Extraction

backend/processor/doc_process.py

Extracts plain text and core properties from .docx/.doc files using python-docx.

● DocxProcessor.process

```
process(self, file_path: str) -> tuple[str, dict]
```

Opens the Word document, joins paragraph texts with newlines, reads core_properties (author, created, modified). Returns (text, metadata).

PARAMETERS

file_path — Absolute path to DOCX

RETURNS

tuple — (str: paragraph text, dict: author/created/modified)

■ *Raises ValueError on error.*

text_process.py — Plain Text Reader

backend/processor/text_process.py

Reads .txt, .csv, and .md files as UTF-8 plain text.

● TextProcessor.process

```
process(self, file_path: str) -> str
```

Opens the file in text mode, reads full content, strips whitespace, and returns it.

PARAMETERS

file_path — Absolute path to text file

RETURNS

str — full file contents, stripped

■ *Raises ValueError on read error.*

policy_engine.py — Routing Flag Setter

backend/policy_engine/policy_engine.py

Translates raw metadata signals into routing flags that FileOrganizer reads.

● PolicyEngine.apply_policies

```
apply_policies(self, file_path: str, metadata: dict) -> None
```

Sets three keys in the metadata dict: is_sensitive (bool), sensitive_types (JSON list of PII type names), is_duplicate (from metadata['duplicate']).

PARAMETERS

file_path — Available for future policy rules

metadata — Mutable dict modified in-place

RETURNS

None — mutates metadata

■ *is_sensitive and is_duplicate are read by FileOrganizer to select the destination subfolder.*

file_organizer.py — File Mover

backend/organizer/file_organizer.py

Moves files to the correct subfolder inside the output directory. Priority: Sensitive > Duplicates > category. Handles filename collisions.

● FileOrganizer.__init__

```
__init__(self, output_folder: str = None)
```

Sets the output base folder. Defaults to ~/Downloads/OmniSort.

PARAMETERS

output_folder — Absolute path to the output root

RETURNS

None

● FileOrganizer.organize_file

```
organize_file(self, file_path: str, metadata: dict) -> str
```

Selects subfolder: 'Sensitive' if is_sensitive, 'Duplicates' if is_duplicate, else metadata['category']. Creates the folder, generates a unique path, moves the file with shutil.move(). Returns the final destination path.

PARAMETERS

file_path — Source path

metadata — Dict with is_sensitive, is_duplicate, category

RETURNS

str — absolute destination path after move

- *shutil.move works cross-filesystem.*
- *Destination path used by file_watcher to compute file_size after move.*

● FileOrganizer._unique_path

```
_unique_path(self, folder: str, filename: str) -> str
```

Returns folder/filename if free. If occupied, appends _1, _2, ... until a free path is found. Prevents silent overwrites.

PARAMETERS

folder — Destination directory

filename — Original filename with extension

RETURNS

str — unique destination path, e.g. /OmniSort/Photos/photo_3.jpg

---

metrics.py — Observability Metrics

backend/metrics/metrics.py

Thread-safe in-process metrics store. Sliding 60-second deque window for files/min. All state protected by a single threading.Lock. Exposed as a module-level singleton.

● Metrics.record_file_processed

```
record_file_processed(self, *, is_duplicate: bool = False) -> None
```

Appends current monotonic time to the sliding window, increments total_files, increments duplicates if is_duplicate. Trims window entries older than 60 s.

PARAMETERS

is_duplicate — True when the file was routed to Duplicates/

RETURNS

None

● Metrics.record_ocr_failure

```
record_ocr_failure(self) -> None
```

Increments ocr_failures. Called by file_watcher when OCRExtractor raises.

RETURNS

None

● Metrics.record_classification

```
record_classification(self, elapsed_ms: float) -> None
```

Adds elapsed_ms to the classification time sum and increments the count. Covers the full NLP+LLM classification block.

PARAMETERS

elapsed_ms — Wall-clock ms for the classification block

RETURNS

None

● Metrics.record_llm_call

```
record_llm_call(self, elapsed_ms: float) -> None
```

Increments llm_calls and adds elapsed_ms to the LLM latency sum. Separate from record_classification to isolate OpenAI round-trip time.

PARAMETERS

elapsed_ms — OpenAI API round-trip latency in ms

RETURNS

None

● Metrics.snapshot

```
snapshot(self) -> dict
```

Returns a point-in-time snapshot: files_per_min, total_files, ocr_failures, llm_calls, avg_classification_ms, avg_llm_ms, duplicates, duplicate_rate. All computed under the lock for consistency.

RETURNS

dict — 8-key observability snapshot

■ *Exposed via GET /api/metrics.*

● Metrics._trim_window

```
_trim_window(self, now: float) -> None
```

Pops timestamps older than now-60.0 from the left of the deque. Must be called inside the lock.

PARAMETERS

now — Current time.monotonic() value

RETURNS

None — modifies deque in-place

db.py — SQLite Data Layer

backend/database/db.py

All database interaction. Uses sqlite3 with row_factory=sqlite3.Row. Each function opens its own connection and closes it immediately.

● get_connection

```
get_connection() -> sqlite3.Connection
```

Opens a connection to DB_PATH (database/omnisort.db by default). Sets row_factory to sqlite3.Row for dict-like access.

RETURNS

sqlite3.Connection — caller must close

● init_db

```
init_db() -> None
```

Creates the 'files' table if absent (id, filename, original_path, destination_path, extension, category, file_size, file_hash, is_duplicate, is_sensitive, sensitive_types, embedding, processed_at). Runs ALTER TABLE ADD COLUMN embedding in try/except to migrate existing databases.

RETURNS

None

- *Idempotent — running multiple times is safe.*
- *Called by main.py (via FileWatcher.__init__) and api.py on startup.*

● insert_file

```
insert_file(record: dict) -> None
```

Inserts a new row using named parameter binding (:key). All 11 non-id columns supplied from the record dict built in _process_file.

PARAMETERS

record — Dict: filename, original_path, destination_path, extension, category, file_size, file_hash, is_duplicate, is_sensitive, sensitive_types, embedding

RETURNS

None

- *Called inside the per-hash lock, inside an isolated try/except in file_watcher.*

● get_files

```
get_files(limit: int = 50, offset: int = 0) -> list[dict]
```

Returns paginated file records ordered by processed_at DESC.

PARAMETERS

limit — Max rows

offset — Skip N rows for pagination

RETURNS

list[dict] — file rows

● get_files_with_embeddings

```
get_files_with_embeddings() -> list[dict]
```

Returns all rows where embedding IS NOT NULL. Used by the semantic search endpoint to retrieve candidate files for cosine similarity ranking.

RETURNS

list[dict] — rows with stored embedding vectors

■ *Images, video, and audio have NULL embeddings and are excluded.*

● get_stats

```
get_stats() -> dict
```

Runs COUNT and GROUP BY queries. Returns total, duplicates, sensitive counts, and per-category breakdown. Used by GET /api/stats.

RETURNS

dict — {total, duplicates, sensitive, by_category: {str: int}}

---

embedding.py — Semantic Embedding

backend/embeddings/embedding.py

384-dimensional text embeddings via fastembed (ONNX, BAAI/bge-small-en-v1.5). Works on Python 3.11+ — no PyTorch required. Model loaded lazily and cached. All failures degrade gracefully to [] so file sorting continues uninterrupted.

● _get_model

```
_get_model() -> fastembed.TextEmbedding
```

Lazily loads BAAI/bge-small-en-v1.5 into the module-level _model variable. Downloads ~40 MB ONNX model on first call; subsequent calls return the cached instance. Import is deferred so the module loads even if fastembed is not installed.

RETURNS

fastembed.TextEmbedding instance

■ *Thread-safe: Python's GIL serialises the first model load.*

● embed_text

```
embed_text(text: str) -> list[float]
```

Returns a 384-dim embedding for the given text. Truncates to 1,000 chars, calls _get_model().embed(), converts numpy array to Python list. Returns [] on any error.

PARAMETERS

text — Document text — first 1,000 chars used

RETURNS

list[float] — 384-element vector, or [] on failure

■ *[] result means embedding column is NULL in SQLite — file is still sorted correctly.*

■ *Called once per file in _process_file after text extraction.*

● cosine_similarity

```
cosine_similarity(a: list[float], b: list[float]) -> float
```

Computes cosine similarity using numpy dot product / (L2-norm product). Returns 0.0 if either vector is empty or lengths differ.

PARAMETERS

a — Query embedding (from embed_text)

b — Stored file embedding (from DB)

RETURNS

float — similarity in [0.0, 1.0]; 1.0 = identical

■ *Used by GET /api/search to rank files by query relevance.*

api.py — FastAPI REST + WebSocket Server

backend/api/api.py

Exposes OmniSort data over HTTP and WebSocket. CORS is open (*) since the server binds to 127.0.0.1 by default. The WebSocket polls an in-process queue and broadcasts file events to all connected dashboard clients.

● on_startup

```
@app.on_event('startup') on_startup() -> None
```

FastAPI lifecycle hook. Calls db.init_db() to ensure the SQLite schema exists before any request is served.

RETURNS

None

● health

```
GET /api/health -> dict
```

Liveness probe. Returns {'status': 'ok'}. Used by Docker health checks.

RETURNS

{'status': 'ok'}

● stats

```
GET /api/stats -> dict
```

Returns aggregated statistics from db.get_stats(): total, duplicates, sensitive, and per-category counts.

RETURNS

dict — {total, duplicates, sensitive, by_category}

● files

```
GET /api/files?limit=50&offset;=0 -> list[dict]
```

Paginated file history from db.get_files(). The Electron dashboard uses this to populate the file history table on load.

PARAMETERS

limit — Max rows (default 50)

offset — Skip N rows (default 0)

RETURNS

list[dict] — file records

● get_metrics

```
GET /api/metrics -> dict
```

Returns the current observability snapshot from the in-process metrics singleton: files_per_min, total_files, ocr_failures, llm_calls, avg_classification_ms, avg_llm_ms, duplicates, duplicate_rate.

RETURNS

dict — see Metrics.snapshot()

---

search

```
GET /api/search?q=...&limit;=10 -> list[dict]
```

Semantic search. Embeds the query with embed_text(), fetches all files with stored embeddings, computes cosine_similarity for each, sorts descending, returns top N. Each result includes all file columns plus 'score'.

PARAMETERS

q — Natural language search query

limit — Max results (default 10)

RETURNS

list[dict] — file records sorted by score DESC, each with score: float

- *Files without embeddings (NULL) are excluded via get_files_with_embeddings().*
- *Query embedding is generated fresh per request.*

---

websocket_endpoint

```
WS /ws async websocket_endpoint(websocket: WebSocket)
```

Accepts WebSocket connections, polls event_queue.get_nowait() every 200 ms, broadcasts each event as JSON to all active connections. Removes client on disconnect.

PARAMETERS

websocket — FastAPI WebSocket instance

RETURNS

None — runs until client disconnects

- *event_queue is shared with file_watcher._process_file.*
- *200 ms polling balances latency vs CPU usage.*

---

logger.py — Structured Logger

backend/logger/logger.py

Thin wrapper around Python's stdlib logging. Writes to both a file and stdout.

● Logger.__init__

```
__init__(self, log_file: str = 'omnisort.log')
```

Configures logging.basicConfig with INFO level, timestamped format, FileHandler (~/log_file) and StreamHandler (stdout).

PARAMETERS

log_file — Filename written to user home directory

RETURNS

None

■ *Log path: os.path.expanduser('~') / log_file*

● Logger.log

```
log(self, message: str) -> None
```

Logs at INFO level. Used for normal events: 'Watching X', 'Sorted: file -> category'.

PARAMETERS

message — Log message

RETURNS

None

● Logger.error

```
error(self, message: str) -> None
```

Logs at ERROR level. Used for processing failures, DB errors, OCR exceptions.

PARAMETERS

message — Error message

RETURNS

None

● Logger.warn

```
warn(self, message: str) -> None
```

Logs at WARNING level for soft failures and recoverable conditions.

PARAMETERS

message — Warning message

RETURNS

None
